

Computer Science Engineering – Cyber Security

- Topic:** Zero Trust Security for Web Applications in Microservice-Based Environments
- Domain:** Cybersecurity – DevOps
- Members:** Dinesh Kumar.N (ch.en.u4cys21014)
V. Jayaraj (ch.en.u4cys21026)
- Supervisor:** Dr. S. Udhayakumar
CSE– Cybersecurity



Problem Identification

- Traditional authentication and authorization mechanisms often rely on perimeter-based security, which fails to protect against internal threats and lateral movement within containerized environments.
- Increasing use of cloud services and remote work has expanded the attack surface, making it difficult to secure data and applications effectively.
- The complexity and scale of managing permissions and access controls in containerized and microservices architectures expose vulnerabilities.

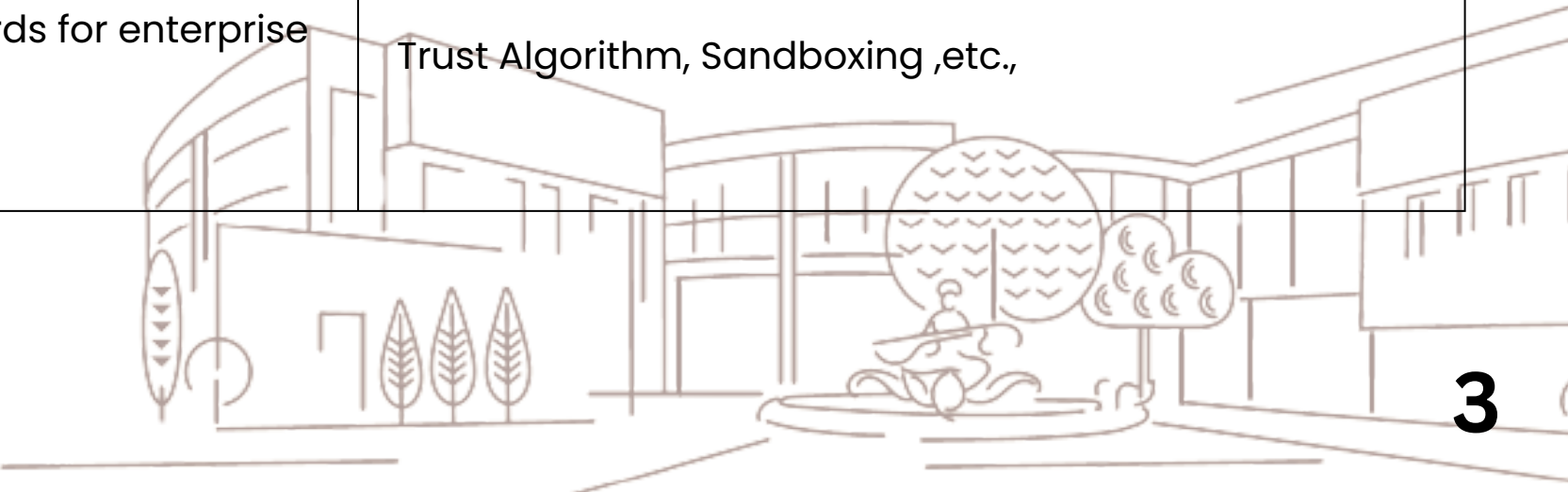
Problem Statement

- The current reliance on perimeter-based security fails to provide the necessary protection for data and applications in the context of expanding cloud services and remote work, necessitating a shift to a Zero Trust Architecture.
- Zero Trust Architecture addresses these gaps by enforcing strict identity verification, continuous monitoring, and least-privilege access, significantly enhancing security in dynamic and distributed container deployments.



Literature Review

S.NO	Title and Author	Objective	Algorithms/Techniques
1	Building A Zero Trust Architecture Using Kubernetes Daniel D'Silva; Dayanand D. Ambawade, published in IEEE: 2021 I2CT	Use containers to build a Zero Trust Achitecture providing authentication with Least Privege	Attribute based access control, JSON Web Token
2)	Establishing a Zero Trust Strategy in Cloud Computing Environment Saima Mehraj; M. Tariq Banday, published in IEEE: 2020 ICCCI	Propose a Zero Trust strategy to enhance trust establishment between customers and cloud service providers	Multi Factor Authentication, Mapping Data Flow, Microsegmentation
3)	Authentication and authorization orchestrator for microservice-based software architectures A. Bánáti1 , E. Kai1l , K. Karóczkai1 and M. Kozlovsky1 MIPRO 2018	Investigating the different types of authentication and authorization methods in order to implement our solution for a healthcare application.	OAuth2, SSO, JWB and OpenID
4)	Securing a Cloud-Native C2 Architecture Using SSO and JWT Ryan Melton, 2021 !IEEE Aerospace Conference	To implement Zero Trust Architecture on Command and Control application named COSMOS in kubernetes cluster	Istio (mTLS), JWT, KeyCloak, SSO
5)	Zero Trust Architecture Scott Rose Oliver Borchert Stu Mitchell Sean Connelly NIST Special Publication 800-207	Define Zero Trust Architecture standards for enterprise organizations	Trust Algorithm, Sandboxing ,etc.,



Project Significance

- Implementing Zero Trust Architecture (ZTA) in containerized environments is crucial for enhancing security.
- By implementing least-privilege access and continuous monitoring, organizations can better protect their sensitive data and applications from evolving threats.
- Containers are inherently ephemeral and dynamic, making traditional perimeter-based security ineffective.
- ZTA benefits organizations, DevOps teams, and IT security professionals by improving security posture, ensuring regulatory compliance, and enabling efficient, secure deployments, thus promoting safer and more resilient infrastructure.

Tech Stack Used

Frontend: React js -- TailwindCSS -- Redux

Backend: Python -- flask -- Node js

DevOps:

- Docker
- Kubernetes – AWS EKS
- Helm Chart
- Git

Database: MongoDB /AWS DynamoDB

ZTA Features: JWT -- mTLS -- Falco -- R8

Android App: React Native -- Java



Objective

- Design and implement a Zero Trust Architecture that enforces strict identity verification and continuous monitoring, ensuring that every access request within containerized environments is authenticated and authorized.
- Develop mechanisms for least-privilege access, ensuring that users and applications have only the permissions necessary for their tasks, minimizing the risk of unauthorized actions.
- Integrate advanced security measures that protect against internal and external threats, ensuring a robust and resilient security posture in dynamic containerized environments.

Scope

To Evaluate current security.

Develop a Zero Trust framework for containers, integrate continuous identity verification and access controls.

Develop a POC for ZTA in web and android applications, conduct testing, provide documentation.

Github Repo: <https://github.com/Dinesh-4320/Zero-Trust-Project>



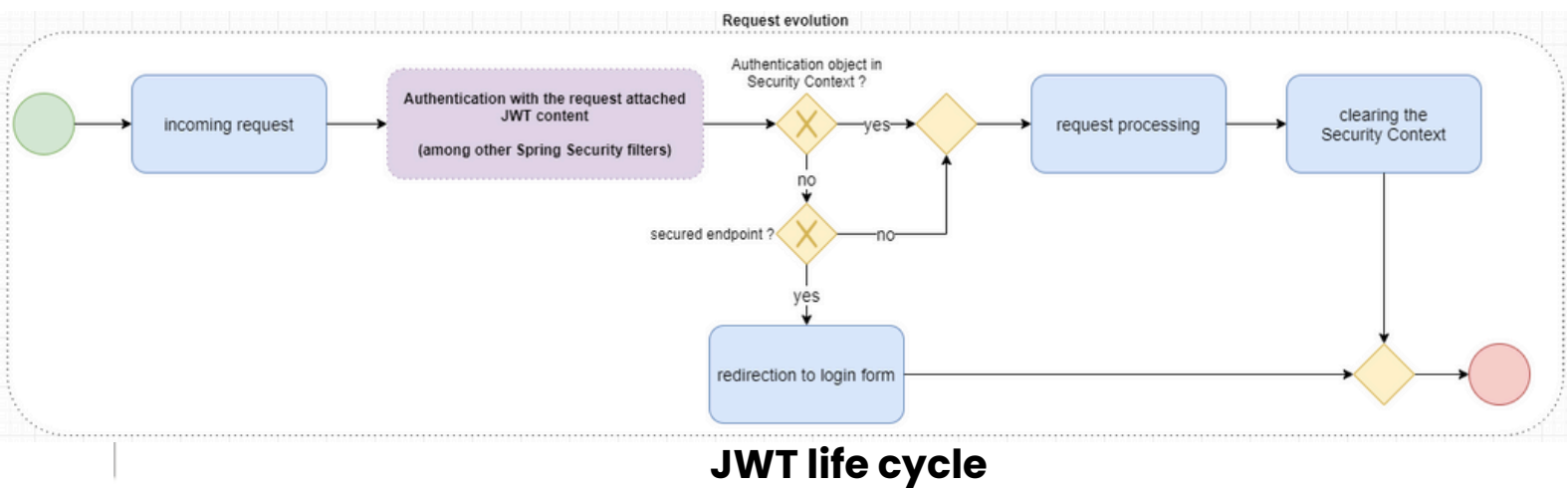
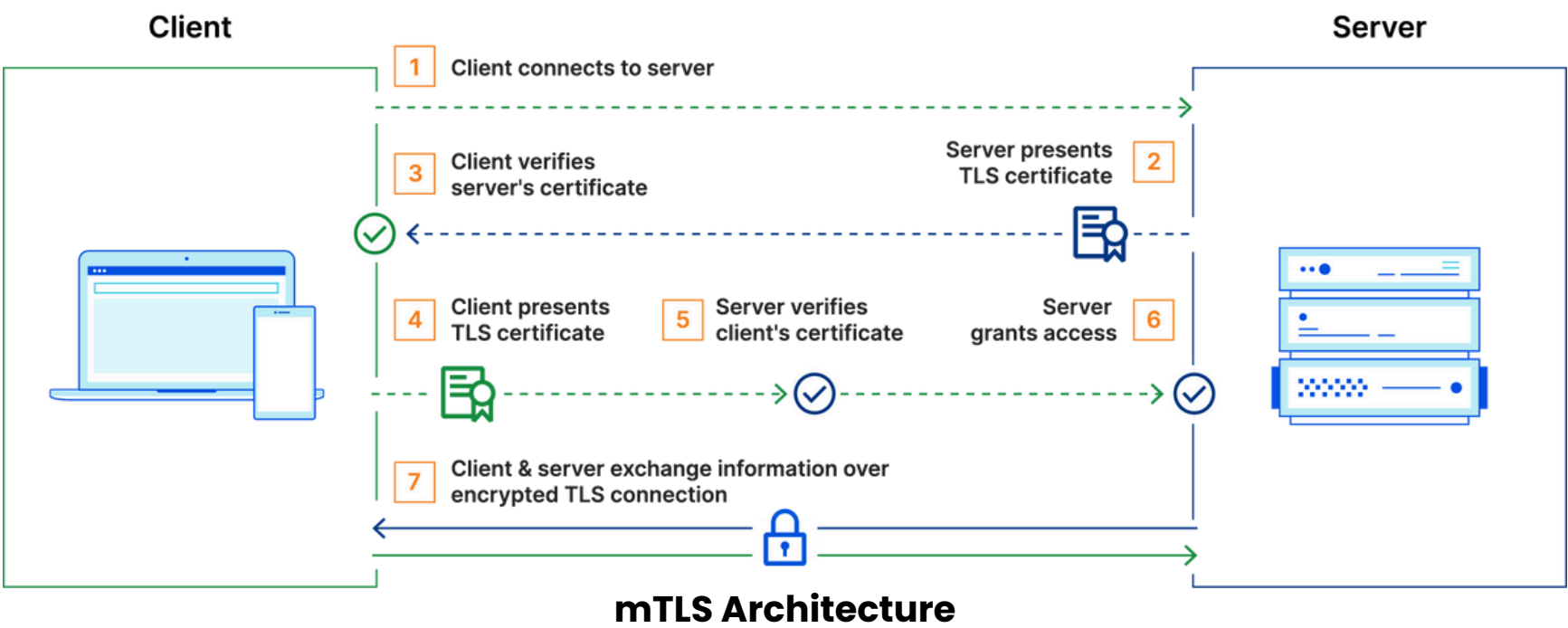
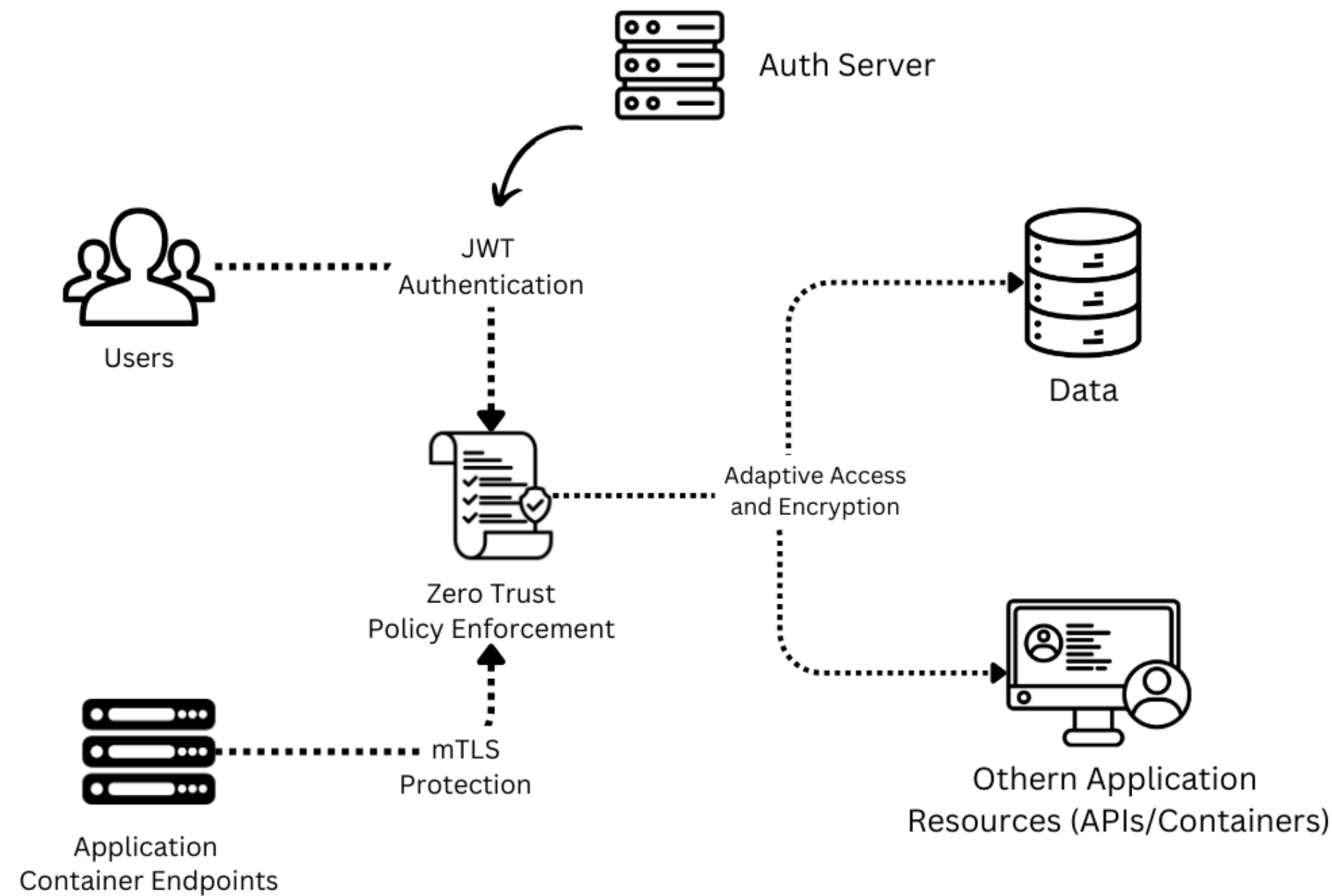
Zero Trust Security for Web Applications in Microservice-Based Environments

N. Dinesh Kumar

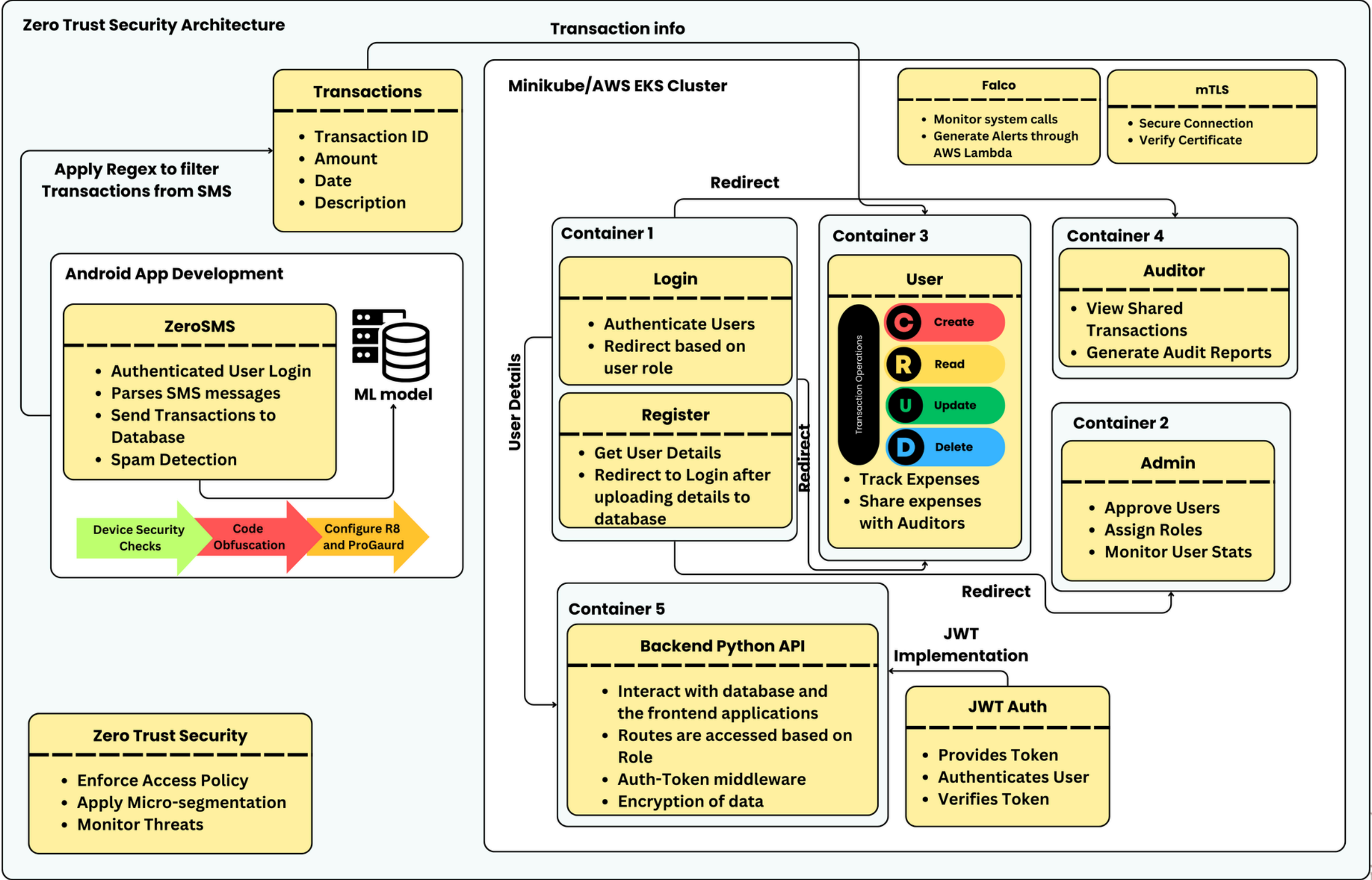
V. Jayaraj



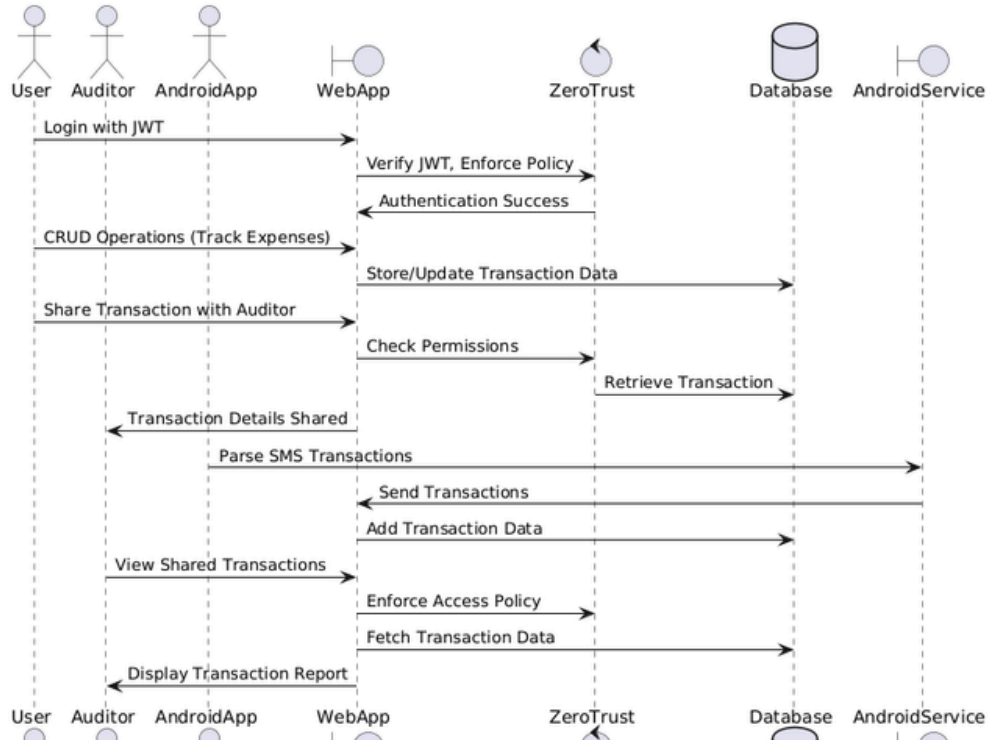
General ZTA Diagram



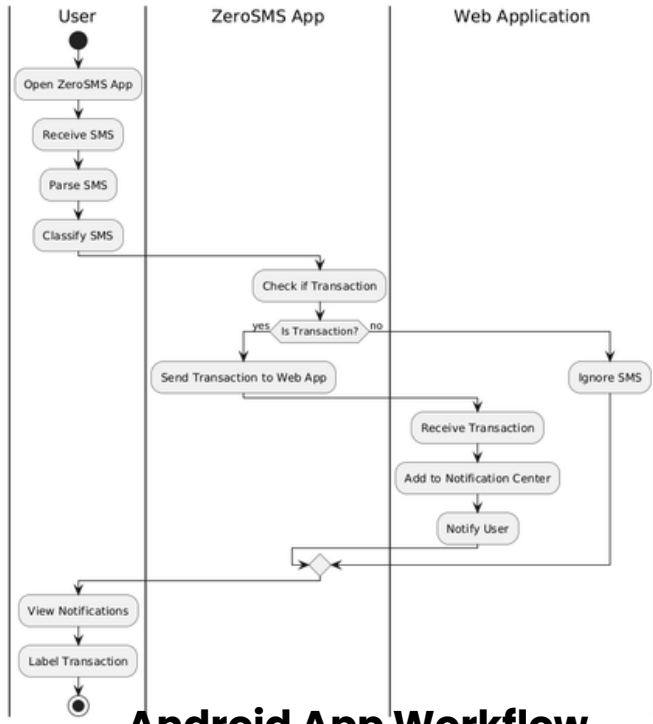
Implementation



Web and Android Application Architecture Diagram



Sequence Diagram



Android App Workflow

Results and Analysis

The web application, deployed on an EKS cluster with node groups, ensures scalability, fault tolerance, and seamless service communication for a smooth user experience. The user dashboard supports secure CRUD operations, while data sharing with auditors provides selective access. Auditors have read-only access to transactions, and the admin dashboard allows dynamic role assignment and access control.

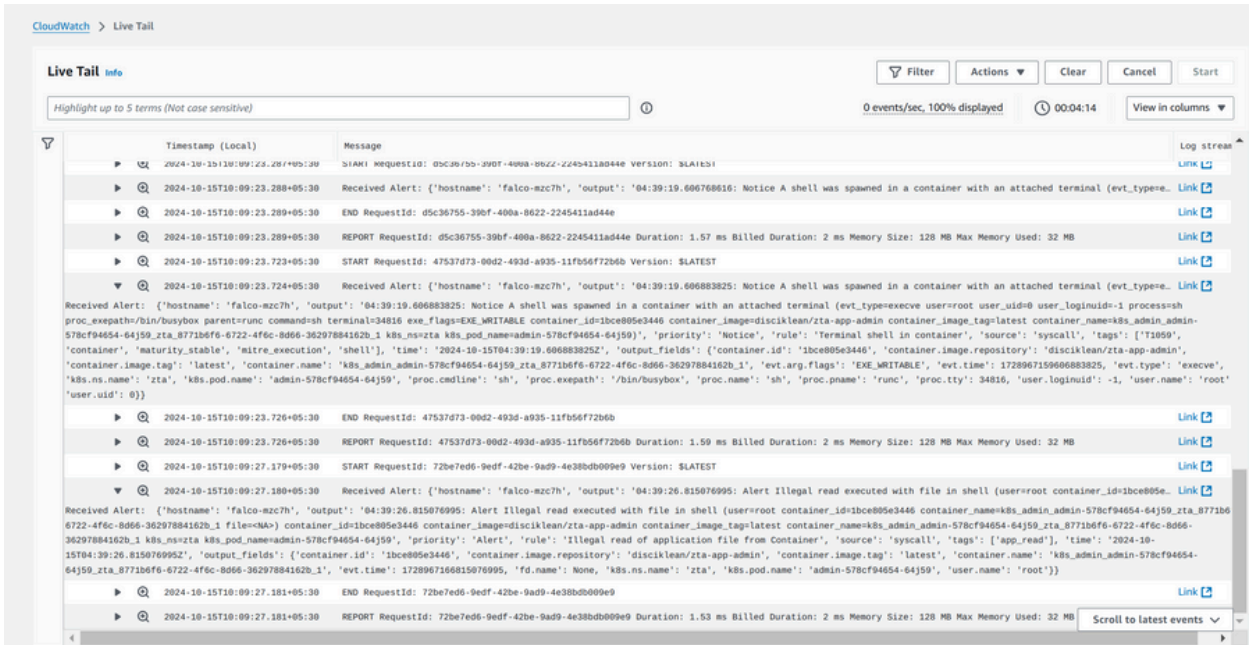
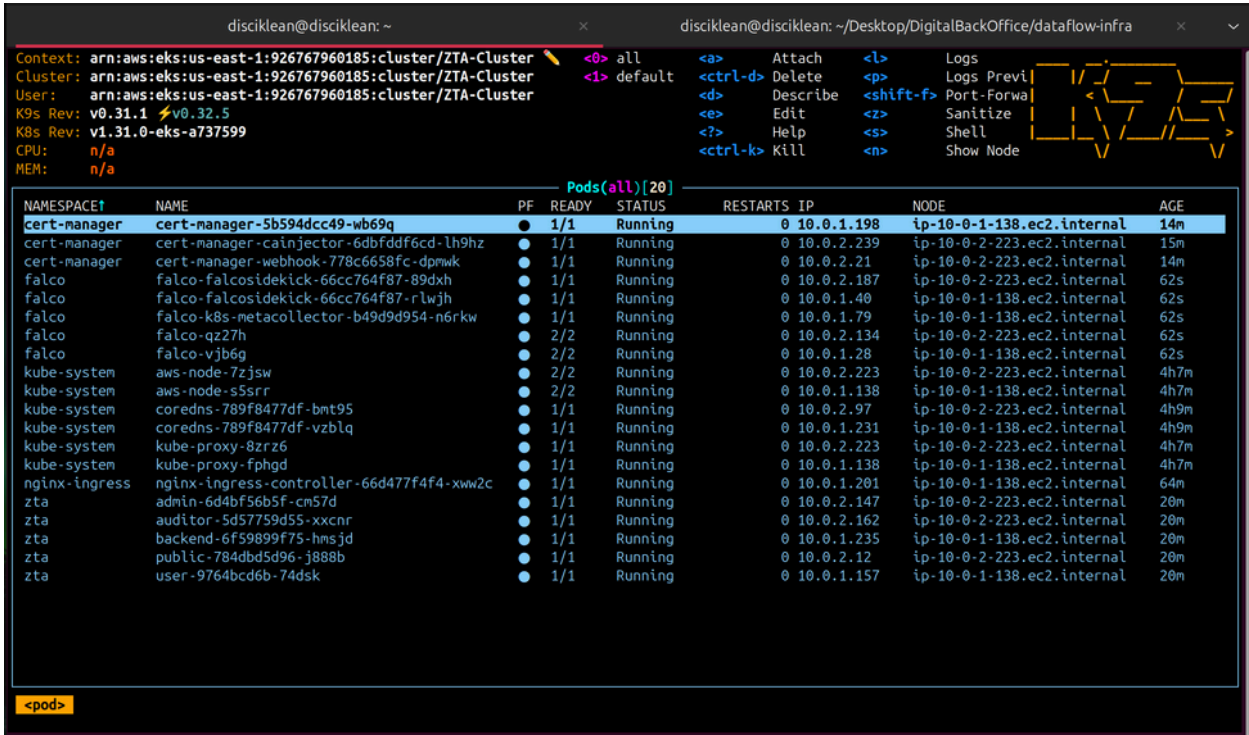
Security is enhanced using AWS Lambda and Falco, which monitor the cluster for anomalies and trigger real-time alerts for threat detection. Session logs offer a comprehensive audit trail, tracking user activities and ensuring compliance with Zero Trust principles by continuously monitoring user actions.

Naive Bayes was the best model with an F1 score of 0.918 and accuracy of 0.978, showing a strong balance of precision and recall. SVM had the highest precision (1.0) and ROC AUC score (0.984), while Logistic Regression also performed well with an F1 score of 0.909. Random Forest and Gradient Boosting had slightly lower F1 scores but maintained high accuracy across all models.

Zero Trust Security for Web Applications in Microservice-Based Environments

N. Dinesh Kumar

V. Jayaraj



Results and Analysis

To prepare for production release, the app leverages the JailMonkey library to detect rooting and unsafe environments, while utilizing R8 and ProGuard tools for code optimization and obfuscation to enhance performance and security. The APK is securely signed with a private key from a trusted keystore, ensuring authorized distribution.

The below figures compares and tests ZeroSMS APK against standard APKs. Due to R8 and ProGuard obfuscation, reverse engineering ZeroSMS is significantly more challenging. A MobSF scan shows ZeroSMS achieving a security score of 57 (Grade B), 83.87% higher than a standard APK, which scored 31 (Grade C). ZeroSMS has SMS permissions and allows "http" traffic for backend connectivity, impacting its security score.

app-1.apk

DeleteDownload ZIP

app-1.apk / sources

..

android/support/v4

androidx

com

kotlin

kotlinx/coroutines

org

ZeroSMS.apk

DeleteDownload ZIP

ZeroSMS.apk / sources

..

A

a

a0

A0/a.java

A1

a1

a2/C0228a.java

A2

a3

```
package com.example.firstapp;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    Button add;
    TextView display;
    Button reset;
    Button sub;
    EditText x;
    EditText y;

    /* access modifiers changed from: protected */
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        this.x = (EditText) findViewById(R.id.et1);
        this.y = (EditText) findViewById(R.id.et2);
        this.add = (Button) findViewById(R.id.b1);
        this.sub = (Button) findViewById(R.id.b2);
        this.reset = (Button) findViewById(R.id.b3);
        this.display = (TextView) findViewById(R.id.t4);
        this.add.setOnClickListener(new View.OnClickListener() {
            public void onClick(View v) {
                MainActivity.this.display.setText(Integer.toString(Integer.parseInt(x.getText().toString()) + Integer.parseInt(y.getText().toString())));
            }
        });
        this.sub.setOnClickListener(new View.OnClickListener() {
            public void onClick(View v) {
                MainActivity.this.display.setText(Integer.toString(Integer.parseInt(x.getText().toString()) - Integer.parseInt(y.getText().toString())));
            }
        });
        this.reset.setOnClickListener(new View.OnClickListener() {
            public void onClick(View v) {
                MainActivity.this.x.setText("");
                MainActivity.this.y.setText("");
                MainActivity.this.display.setText("");
            }
        });
    }
}
```

ZeroSMS.apk

DeleteDownload ZIP

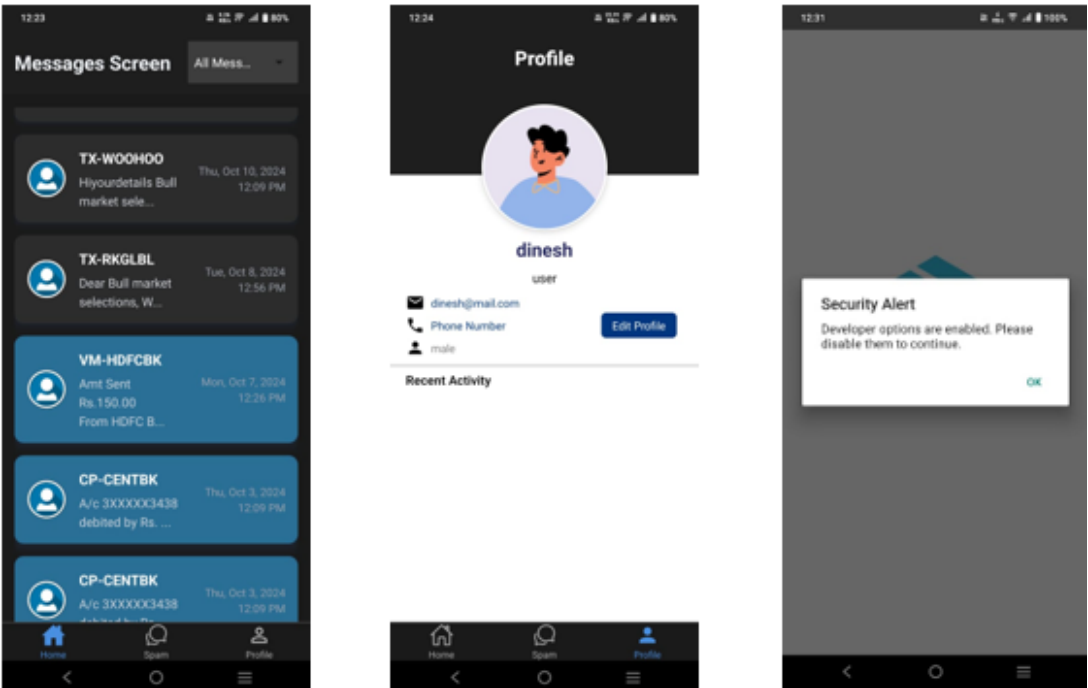
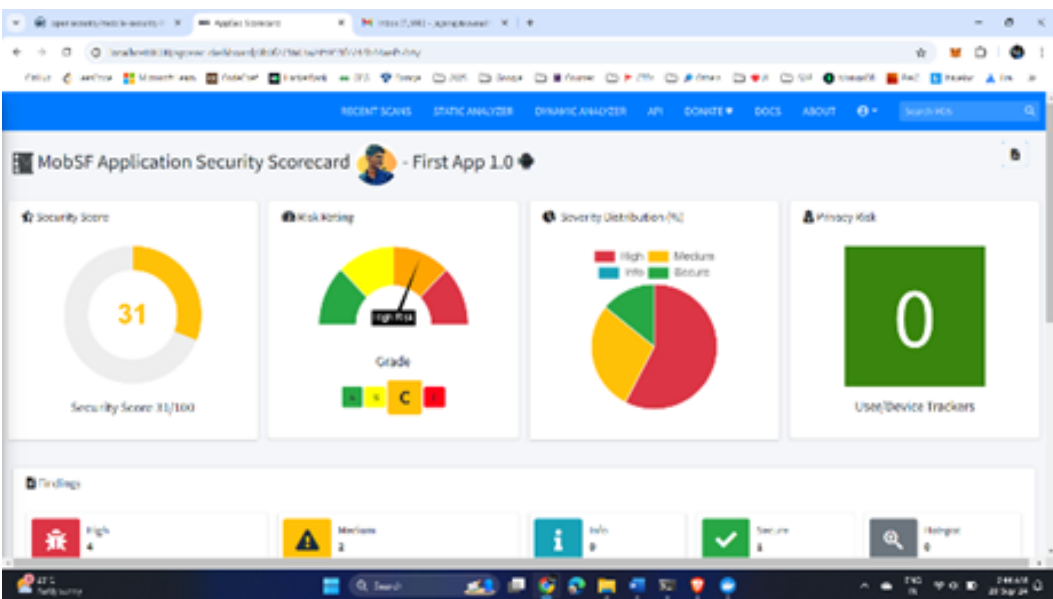
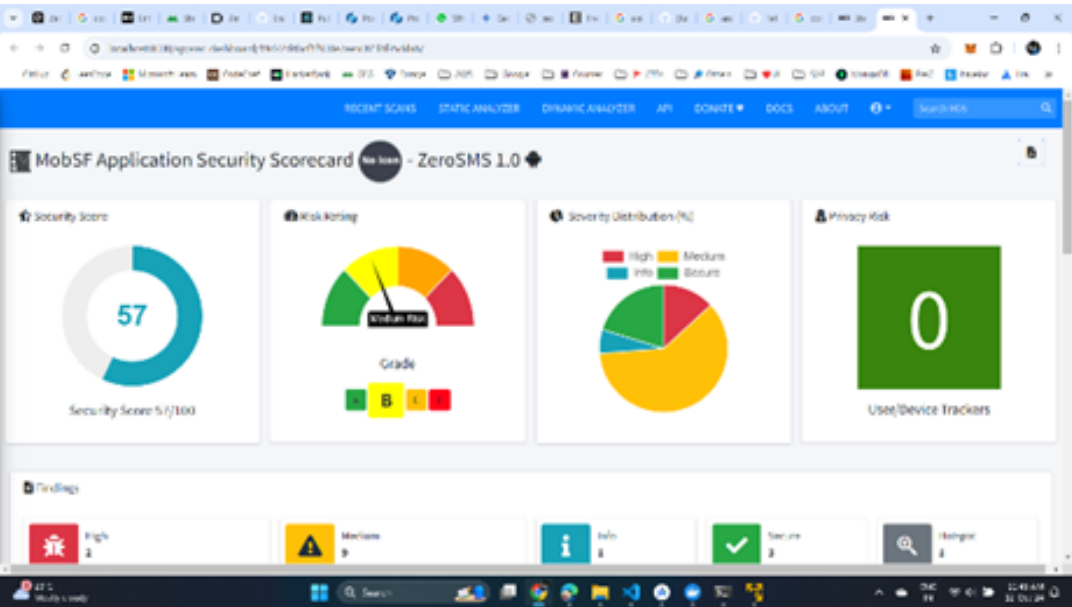
ZeroSMS.apk / sources / e1 / C0522a.java

```
package e1;

import d3.A;
import d3.C0511e;

/* renamed from: e1.a reason: case insensitive filesystem */
public abstract class C0522a {
    public static void a(A a4, Object obj) {
        for (C0511e eVar : a4.r().i()) {
            if (obj.equals(eVar.i().j())) {
                eVar.cancel();
                return;
            }
        }
        for (C0511e eVar2 : a4.r().j()) {
            if (obj.equals(eVar2.i().j())) {
                eVar2.cancel();
                return;
            }
        }
    }
}
```

```
signingConfigs {
    debug {
        storeFile file('debug.keystore')
        storePassword 'android'
        keyAlias 'androiddebugkey'
        keyPassword 'android'
    }
    release {
        if (project.hasProperty('MYAPP_UPLOAD_STORE_FILE')) {
            storeFile file(MYAPP_UPLOAD_STORE_FILE)
            storePassword MYAPP_UPLOAD_STORE_PASSWORD
            keyAlias MYAPP_UPLOAD_KEY_ALIAS
            keyPassword MYAPP_UPLOAD_KEY_PASSWORD
        }
    }
}
buildTypes {
    debug {
        signingConfig signingConfigs.debug
    }
    release {
        // Caution! In production, you need to generate your own keystore file.
        // see https://reactnative.dev/docs/signed-apk-android.
        signingConfig signingConfigs.release
        debuggable false
        shrinkResources true
        minifyEnabled true
        proguardFiles getDefaultProguardFile("proguard-android.txt"), "proguard-rules.pro"
    }
}
```



Zero Trust Security for Web Applications in Microservice-Based Environments
N. Dinesh Kumar
V. Jayaraj



Conclusion and Future Work

This project demonstrated the application of Zero Trust principles to enhance security in containerized environments through a proof of concept (PoC) that included a secure web application, Python API, and Android app. Key strategies such as JWT authentication, continuous monitoring, root detection, mTLS within containers, and Role-Based Access Control (RBAC) were implemented. The outcomes emphasize the importance of trust verification at every layer and proactive monitoring to significantly mitigate security risks in modern containerized systems.

Future efforts can focus on integrating advanced threat detection using machine learning to identify anomalies in containerized applications. Automating security audits and incorporating continuous vulnerability scanning in CI/CD pipelines will strengthen ongoing security assessments. Additionally, providing Zero Trust training for developers will promote a security-first mindset. Real-world deployments of these solutions in varied organizational settings would offer insights into improving security measures and adapting strategies to evolving cyber threats.



References

- [1] O. E. Imokhai, T. E. Emmanuel, A. A. Joshua, E. S. Emakhu, and S. Adebisi, "Zero Trust Architecture: Trend and Impact on Information Security," International Journal of Emerging Technology and Advanced Engineering, vol. 12, no. 7, pp. 87–92, July 2022.
- [2] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, NIST Special Publication 800–207 Zero Trust Architecture. Gaithersburg, MD: National Institute of Standards and Technology (NIST), 2020.
- [3] Y. He, D. Huang, L. Chen, Y. Ni, and X. Ma, "A Survey on Zero Trust Architecture: Challenges and Future Trends," Wireless Communications and Mobile Computing, vol. 2022, Article ID 6476274, 13 pages, 2022. DOI: <https://doi.org/10.1155/2022/6476274>.
- [4] A. Mustyala and S. Tatineni, "Advanced Security Mechanisms in Kubernetes: Isolation and Access Control Strategies," ESP Journal of Engineering & Technology Advancements, vol. 1, no. 2, pp. 45–52, 2021.
- [5] B. Pranoto, "Threat Mitigation in Containerized Environments," Applied Research in Artificial Intelligence and Cloud Computing, vol. 6, no. 8, pp. 142–151, 2023.
- [6] S. Bagheri, H. Kermabon-Bobinnec, S. Majumdar, Y. Jarraya, L. Wang, and M. Pourzandi, "Warping the Defence Timeline: Non-disruptive Proactive Attack Mitigation for Kubernetes Clusters," in Proc. IEEE International Conference on Communications (ICC 2023), Rome, Italy, 28 May – 01 June, 2023, pp. 567–574.
- [7] D. D'Silva and D. D. Ambawade, "Building a Zero Trust Architecture Using Kubernetes," in Proc. 2021 6th International Conference for Convergence in Technology (I2CT), Pune, India, Apr. 2021, pp. 1–5.
- [8] P. Varalakshmi, B. Guhan, P. V. Siva, T. Dhanush, and K. Saktheeswaran, "Improvising JSON Web Token Authentication in SDN," in Proc. 2022 International Conference on Communication, Computing and Internet of Things (IC3IoT), Mar. 2022. DOI: 10.1109/IC3IoT53935.2022.9767873.

Zero Trust Security for Web Applications in Microservice-Based Environments

N. Dinesh Kumar

V. Jayaraj

